POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

A lightweight canonicalization engine in C

WILLIAM ZHANG
Département de Electrical Engineering

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Electrical Engineering

July 2026

© William Zhang, 2026.

POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

Ce mémoire intitulé :

A lightweight canonicalization engine in C

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Danielle DUBOIS, présidente
Sébastien VIGOT, membre et directeur de recherche
Marie COUTURE, membre et codirectrice de recherche
Jean TREMBLAY, membre
Joseph BROWN, membre externe

DEDICATION

To all the past, present and future CVXPY contributors

ACKNOWLEDGEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE OF CONTENTS

DEDICATION iii

ACKNOWLEDGEMENTS iv

RÉSUMÉ v

ABSTRACT vi

LIST OF TABLES ix

LIST OF FIGURES x

LIST OF SYMBOLS AND ACRONYMS xii

CHAPTER 1 INTRODUCTION 1

CHAPTER 2 LITERATURE REVIEW 5
2.1 Canonicalization 6
2.2 Matrix stuffing 10
2.3 Coefficient extraction 14
2.4 Computing derivatives 16
2.5 Sparse matrices 19

CHAPTER 3 A lightweight canonicalization engine 22
3.1 Keeping parameters, splitting the walk 22
3.2 Fundamental data structures 23
3.3 Atom implementations 25
3.4 Problem class 28
3.5 Parameter updates 28
3.6 CVXPY interface 28

CHAPTER 4 Benchmark results 30
4.1 Benchmark Setup 30
4.2 Results across the benchmark suite 31
4.3 Scaling with problem size 32
4.3.1 Parametric least squares 33
4.3.2 Factor-covariance portfolio 34
4.3.3 Energy-storage scheduling 35

CHAPTER 5 CONCLUSION 39
5.1 Summary of Works 39
5.2 Limitations 39
5.3 Future Research 39

REFERENCES 40

LIST OF TABLES

Table 4.1 Cold compilation time: a fresh problem formed and canonicalized from scratch, C engine versus CVXPY's `CPP` backend (22 runnable benchmarks). Ratio = C engine / `CPP`; values below 1 favor the engine. . . 31

Table 4.2 Parametric re-compilation time on a warm cached chain: parameter values are reassigned and the problem data rebuilt (the engine's target regime). Ratio = C engine / `CPP`. 32

LIST OF FIGURES

Figure 1.1 Example of expression tree for the function `log(|Ax-b|)`. 2

Figure 2.1 Users write functions in Python code that are restricted to the atom library. Variables are labeled with a circle, and parameters with a square. 9

Figure 2.2 After the epigraph transformation reduction, the constraints and the objective are affine functions of the variables. 10

Figure 2.3 Matrix stuffing of problem (2.4) by a recursive walk of the conic-form expression trees of 2.2: each node's matrix is obtained by applying a linear operation to the matrices of its children. 12

Figure 2.4 The two compilation paths on the objective of problem (2.4): the DPP path caches the parameter map Q once and re-solves with a matrix-vector product, while the `ignore_dpp` path evaluates the parameters up front and re-canonicalizes from scratch. 13

Figure 3.1 The engine's two passes over the objective tree of 2.2, with the parameter leaf μ kept across both. 23

Figure 3.2 The two passes on the $Fx - g$ subtree of the constraint tree of 2.3. The constant F enters through the matmul atom, whose block $-F$ has a sparsity pattern fixed once; across solves only the parameter g changes. 24

Figure 4.1 Parametric least squares, cold-start: full canonicalization from scratch versus problem size n ($m = 3n$). The C engine is $\approx 4\times$ faster across the whole range; both backends scale as $\sim n^{2.1}$. 34

Figure 4.2 Parametric least squares, warm-restart: rebuild after reassigning A and b. With no DPP tensor to reuse, `CPP` pays its full cold-start cost again, while the C engine only re-evaluates values on the cached sparsity pattern—a 10–12$\times$ speed-up. 35

Figure 4.3 Parametric least squares, peak process memory (resident-set high-water mark). The backends are within $\sim 30\%$ of each other; both grow as $\sim n^{1.6}$, dominated by the shared interpreter and the dense data matrix A. 36

Figure 4.4 Factor-covariance portfolio, cold-start versus number of assets n ($m =$ 100 factors). The engine leads at small n but its advantage shrinks with size—it scales as $\sim n^{1.2}$ against CPP's $\sim n^{0.9}$, reaching near-parity by $n = 8000$. 37

Figure 4.5 Factor-covariance portfolio, warm-restart after reassigning γ and $L_{\max}$. `CPP` re-canonicalizes in essentially its cold-start time; the engine re-extracts on the cached pattern, a stable 12–13× speed-up across the range. 37

Figure 4.6 Factor-covariance portfolio, peak process memory. The backends are within $\sim$15%; both grow slowly ($\sim n^{0.35}$), reflecting the low-rank factor structure rather than a dense operator. 38

Figure 4.7 Energy-storage scheduling: scaling of cold-start, warm-restart, and peak memory with horizon length T (*placeholder curves; data pending the size-sweep run*). 38

LIST OF SYMBOLS AND ACRONYMS

AD	Automatic Differentiation
AML	Algebraic Modeling Language
ASA	Affine-Solver-Affine
CSC	Compressed Sparse Column
CSR	Compressed Sparse Row
DAG	Directed Acyclic Graph
DCP	Disciplined Convex Programming
DPP	Disciplined Parameterized Programming
LP	Linear Program
MOI	MathOptInterface
MPS	Mathematical Programming System
NLP	Nonlinear Program
QP	Quadratic Program
SOCP	Second-Order Cone Program
SDP	Semidefinite Program

CHAPTER 1 INTRODUCTION

Domain specific modeling languages for convex optimization such as CVXPY [1] and YALMIP [2] allow users to formulate optimization problems in a natural mathematical syntax. Through a process called *canonicalization*, these high-level problem descriptions are converted to a *standard conic form* used by many general-purpose convex solvers

$$\begin{array}{ll} \text{minimize} & c^T x + d \\ \text{subject to} & b - Ax \in K \end{array} \tag{1.1}$$

where $x \in \mathbf{R}^n$ is the decision variables, and the *problem data* are the matrix $A \in \mathbf{R}^{m \times n}$, the vectors $b \in \mathbf{R}^m$ and $c \in \mathbf{R}^n$, the scalar $d \in \mathbf{R}$ and the non-empty closed convex cone $K \subseteq \mathbf{R}^m$. Almost all of optimization practice can be divided into two groups, *modeling* and *solving*. The modeler's role is to formulate problems based on real-world business or engineering applications. The solver's role is to find an optimal solution given the data of a problem instance. Modeling languages serve as the communication bridge between these two groups. When improvements in solvers are made, modeling languages must adapt to allow users to benefit from them. Conversely, it is through natural formulations that modelers can express their requirements and help motivate solver improvements.

While the time taken to solve an optimization problem is often dominated by the solver, the time taken to *canonicalize* (or *compile*) a problem can also be significant, especially for large problems. In the case of CVXPY, the *compilation time* largely lies in *matrix stuffing*, the final step of canonicalization where symbolic expression trees are converted to explicit numerical matrices. Other modeling languages take different approaches to form the problem data, we review these *coefficient extraction* approaches in §2.3.

Atoms. Modeling languages for convex optimization are built on top of a library of functions which can be used to describe problems. These functions are known as *atoms* in CVXPY since they are the fundamental building blocks of the modeling language [3]. We write ϕ for a general atom from this library. Atoms have known attributes such as curvature, monotonicity, and sign, for example, .

Expression trees. An expression is recursively defined as an atom evaluated at a *subexpression*. The subexpression can either be a variable, a parameter, or another expression. Similarly to atoms, expressions can also be classified by curvature, monotonicity, and sign

which are determined by simple *composition rules* [4]. An expression tree whose atoms are all affine is known as an *affine expression tree*.

Parameterized conic form. The data inputs in a problem instance can either be fixed or varying. In CVXPY, varying data are referred to as *parameters* and fixed data are referred to as *constants*. Parameters are useful for specifying a *family* of problems where the structure is fixed [5]. A *parameterized cone program* extends the conic form in (1.1) by letting the problem data depend on a parameter vector $\theta \in \mathbf{R}^p$:

$$\begin{array}{ll} \text{minimize} & c(\theta)^T x + d(\theta) \\ \text{subject to} & b(\theta) - A(\theta)x \in \mathcal{K}. \end{array} \tag{1.2}$$

When c, d, b and A are *affine with respect to* θ, we can represent the problem data parameter dependence as

$$\begin{bmatrix} c \\ d \end{bmatrix} = Q \cdot \tilde{\theta}, \qquad \text{and} \qquad [A \quad b] = \sum_{i=1}^{p+1} \mathcal{R}[:,:,i] \cdot \tilde{\theta}_i, \tag{1.3}$$

where $Q \in \mathbf{R}^{(n+1)\times(p+1)}$, $\mathcal{R} \in \mathbf{R}^{m\times(n+1)\times(p+1)}$ and $\tilde{\theta} = (\theta, 1) \in \mathbf{R}^{p+1}$. When $A(\theta)$ is sparse, we also require that its structure or *sparsity pattern* must be fixed throughout the different iterations. We refer to Q as the objective *parameter map*, $\mathcal{R}$ as the constraint parameter map, and $\tilde{\theta}$ as the extended parameter vector. The equation for the constraint map is known as a *tensor* product, a generalization of matrix-vector product in higher dimensions. This notation is required to output a constraint matrix from a three dimensional tensor.

Matrix stuffing. Although the original user specified problem formulation may contain nonlinearities, after conversion to standard form, the *canonicalized problem* will only contain affine expressions (with respect to both parameters and variables). Concretely, matrix stuffing refers to the task of forming the constraint parameter map, as the objective parameter

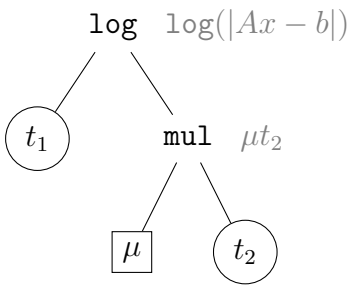


Figure 1.1 Example of expression tree for the function `log(|Ax-b|)`.

map is a scalar special case of it. Once the parameter maps are formed, the parameter dependence of the model is *cached* and doesn't rely on the expression tree representation anymore. Subsequent problem instances will only need to update θ and apply the maps to get the new problem data. We review current CVXPY matrix stuffing approaches and give examples of how the parameter maps are formed in §2.2.

Caching versus recompilation. Once a problem is parameterized, there are two broad strategies for handling repeated solves. The default approach in CVXPY, enabled by the DPP composition rules, forms the parameter maps once and *caches* them; subsequent solves merely apply the maps to a new θ. This is fast to re-solve but requires the problem to be DPP-compliant and stores the large three-dimensional tensor $\mathcal{R}$ in memory. The alternative, selected in CVXPY with `ignore_dpp=True`, skips the parameter maps entirely: parameters are treated as ordinary constants and the problem is re-canonicalized from scratch on every solve. This removes the DPP restriction and the tensor, but pays the full matrix-stuffing cost each time. Our method occupies the middle ground: like the `ignore_dpp` path it forms no parameter tensor and imposes no DPP restriction, but it caches the *sparsity pattern* of the problem data once and only refills the numerical values on each solve, recovering the amortized speed of the DPP path.

Contributions. We make the following contributions:

1. We propose an alternative method to matrix stuffing that doesn't rely on forming parameter maps. Instead, we treat matrix stuffing as a sparse Jacobian evaluation which naturally separates the symbolic structure of the Jacobian from its numerical values. This generalizes the `ignore_dpp` path—it applies to the same broad class of problems—while caching the sparsity pattern rather than re-deriving it on every solve.

2. We use the affine atoms from the `SparseDiffengine` package and its Python bindings to implement a canonicalization backend for CVXPY.

3. We benchmark our method against existing approaches and make improvements to CVXPY's codebase.

All code is open source and available at `https://github.com/cvxpy/cvxpy`.

Thesis outline. The rest of this thesis begins with a review of the literature on modeling languages, canonicalization, and matrix stuffing in §2. In §3 we describe a set of relevant

affine atoms and their implementation in a sparse differentiation package. In §4 we present benchmark results comparing our method to existing approaches. Finally, in §5 we summarize our contributions and discuss future improvements to the canonicalization backend.

CHAPTER 2 LITERATURE REVIEW

History and background Since the early days of linear programming (LP), practitioners have needed to translate mathematical formulations into data that solvers can process. At first, this meant encoding each coefficient of the problem matrices by hand, a tedious and error-prone process [6]. *Matrix generators* emerged in the late 1960s to automate this task, computing the coefficients through explicit loops and writing them out in solver-readable formats like MPS [7, 8].

Still, a gap remained: modelers think in terms of sets, indices, and algebraic expressions, while procedural code constructs the coefficient matrices entry by entry. This mismatch made models difficult to verify, modify, and communicate [9]. *Algebraic modeling languages* such as AMPL [10], AIMMS [11] and GAMS [12] closed this gap by accepting problem descriptions that closely follow the math. For a fuller account of the early history of modeling languages, see [13].

More recently, modeling languages embedded in general-purpose programming languages have gained popularity, such as Pyomo [14,15] in Python, JuMP [16] in Julia, and CasADi [17] in C++. Other domain-specific modeling languages specialized for convex optimization [18] are built on *disciplined convex programming* (DCP), a set of composition rules under which the convexity of a problem can be verified automatically [3, 4]. These DCP-based languages include CVX [19] in MATLAB, CVXR [20] in R, CVXPY [1] in Python, and Convex.jl [21] in Julia.

Structure of the chapter The first two sections focus on CVXPY: it is the most actively maintained DCP implementation, and the work of this thesis aims to be integrated into it. §2.1 covers the *frontend* of canonicalization, which rewrites the user's functional form into a functional conic form of purely affine and quadratic expressions. §2.2 covers its *backend*, *matrix stuffing*, which assembles the problem data from this functional conic form.

§2.3 presents the embedded modeling languages JuMP and CasADi as well as the algebraic language AMPL, and compares how each one extracts the problem data coefficients with the approach taken in CVXPY. Since these languages treat coefficient extraction as a sparse Jacobian computation, §2.4 reviews methods for computing sparse derivatives. Finally, §2.5 covers the sparse matrix formats used to store the problem data, along with some useful algorithms for manipulating them.

2.1 Canonicalization

CVXPY is a DCP-based modeling language embedded in Python [1], which began in 2013 as an implementation of the modeling principles of CVX [19]. Three fundamental design decisions differed from CVX at the beginning. The first is modularity: a few core data structures are shared by separate components for verification, canonicalization, matrix stuffing, and solver interfaces. The second is the expression tree as the central data structure, which is formed automatically from user code through Python's operator overloading. The third is support for parameters: since canonicalization never looks at the values of constants, much of the compilation work can be reused when a problem is solved repeatedly.

These ideas matured with version 1.0, which redesigned canonicalization as a general reduction-based rewriting system [22]. The new design enabled the addition of other disciplined grammars [23–25], and the early parameter support grew into disciplined parameterized programming (DPP), a restricted grammar under which canonicalized programs can be updated efficiently [26].

The open source project has since grown through contributions from many other enthusiastic developers; a recent paper surveys the new canonicalization and modeling features introduced between versions 1.1 and 1.9 [27].

Functional form. A user most naturally expresses a CVXPY problem in *functional* form,

$$\begin{array}{lll} \text{minimize} & f_0(x;\theta) & \\ \text{subject to} & f_i(x;\theta) \le g_i(x;\theta), & i = 1,\dots,m_1, \\ & h_i(x;\theta) = j_i(x;\theta), & i = 1,\dots,m_2, \end{array} \tag{2.1}$$

where the optimization variables $x_1,\dots,x_k$ are collected into a single vector $x \in \mathbf{R}^n$, and likewise the parameters $\theta_1,\dots,\theta_l$ into $\theta \in \mathbf{R}^p$ [18]. We call (2.1) a problem *instance* when θ is fixed to a numerical value, and a problem *family* when θ is left symbolic. Internally, CVXPY represents each function in (2.1) as an expression tree.

Disciplined convex programming. When f_0 and all f_i are convex, all g_i are concave, and all h_i and j_i are affine, the functional form defines a convex optimization problem. To certify convexity, CVXPY uses the DCP composition rules to classify the curvature of each function. Once all functions are classified, a problem is *DCP-compliant* if the objective and each constraint have curvatures compatible with a convex problem: a convex objective, convex $\le$ concave inequalities, and affine equalities.

Otherwise, if a curvature is incompatible (*e.g.* f_0 is concave) or cannot be determined, CVXPY raises an error and terminates. Because these rules are only sufficient for convexity, a problem that is genuinely convex but not expressed in a recognized form may still be rejected.

Composition rules. The composition $\phi(f_1(x), f_2(x), \ldots, f_k(x))$ of an atom ϕ with subexpressions f_i is convex when ϕ is convex and, for each argument $i \in \{1, \ldots, k\}$, one of the following holds:

- ϕ is non-decreasing in argument i and f_i is convex, or
- ϕ is non-increasing in argument i and f_i is concave, or
- f_i is affine.

The same results hold for concave functions by swapping convex and concave. Affine expressions require ϕ and all of its arguments to be affine [3].

Functional conic form. Once a problem is verified to be DCP-compliant, the frontend of canonicalization rewrites its functional form (2.1) into an equivalent *functional conic form*

$$\begin{array}{ll} \text{minimize} & f_0(x;\theta) \\ \text{subject to} & \ell_i(x;\theta) \in \mathcal{K}_i, \quad i = 1, \ldots, m, \end{array} \tag{2.2}$$

where f_0 is affine or quadratic, each ℓ_i is affine, and each $\mathcal{K}_i$ is one of the standard convex cones: the zero, nonnegative, second-order, power, exponential, and positive semidefinite cones.

Reductions. This rewriting is carried out as a sequence of *reductions*. Each reduction provides an `apply` step that converts a problem into an equivalent one, and a `retrieve` step that recovers a solution of the original problem from a solution of the rewritten one [22].

A first example is `FlipObjective`, which turns a maximization problem into a minimization one by flipping the sign of the objective; its `retrieve` step simply flips the sign back.

A second example, `CvxAttr2Constr`, handles variable attributes such as `nonneg` or bounds. When the target solver supports them natively, CVXPY passes them directly to the solver; otherwise it rewrites them as explicit constraints, and the `retrieve` step copies those constraints' dual values back onto the variable bounds.

Conic reformulation. The main reduction, `Dcp2Cone`, rewrites every nonlinear atom into conic constraints. It first rewrites the problem into *Smith form*, replacing each nonlinear atom with a new variable coupled to it by an equality constraint; every atom then acts only on affine arguments [28]. For example, $\|Fx - g\|_2$ is replaced by a new variable t with $t = \|Fx - g\|_2$, leaving the norm applied to the affine argument $Fx - g$. For a convex atom ϕ, this auxiliary variable is called its *epigraph variable* t, defined by $\phi(x) = t$.

Because a DCP-compliant problem only ever minimizes or upper-bounds the value of ϕ, relaxing this equality to the inequality $\phi(x) \leq t$ is *lossless*. A concave atom instead uses a *hypograph* variable and the reversed inequality $\phi(x) \geq t$.

Each epigraph inequality is then replaced by the atom's *graph implementation*: the membership of an affine function of (x, t) in a convex cone [29]. For instance, the inequality $\|x\|_2 \leq t$ is exactly the second-order cone constraint

$$(t, x) \in \mathcal{K}_{\text{soc}}. \tag{2.3}$$

The absolute value $|x|$ is another example: though nonsmooth at the origin, its epigraph $|x| \leq t$ is simply the pair of affine inequalities $-t \leq x \leq t$. The reformulation therefore guarantees that all functions are affine and lie in convex cones, exactly the functional conic form of (2.2).

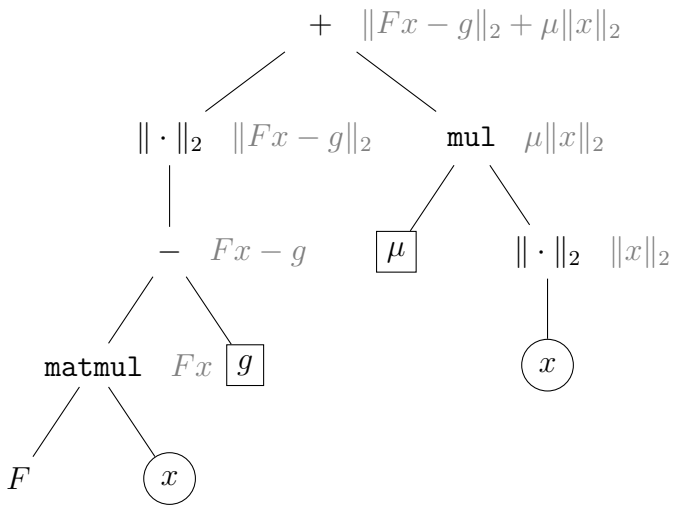
Example of canonicalization. To show a concrete example of canonicalization, we consider a simple fitting with regularization problem

$$\begin{array}{ll} \text{minimize} & \|Fx - g\|_2 + \mu\|x\|_2 \\ \text{subject to} & x \succeq 0 \end{array} \tag{2.4}$$

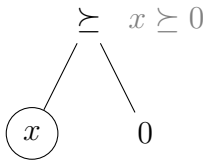
with variable $x \in \mathbf{R}^n$, fixed problem data $F \in \mathbf{R}^{m\times n}$, and parameters $g \in \mathbf{R}^m$ and $\mu \geq 0$. The problem formulation in CVXPY code is

```
import cvxpy as cp
import numpy as np
F = np.random.randn(m, n)
x = cp.Variable(n)
g = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)
objective = cp.Minimize(cp.norm(F @ x - g) + mu * cp.norm(x))
prob = cp.Problem(objective, [x >= 0])
```

where we also use the numerical computation library `NumPy` [30] for the data arrays. The expression tree representation of the problem is shown in 2.1.



(a) Expression tree of the objective



(b) Expression tree of the constraints

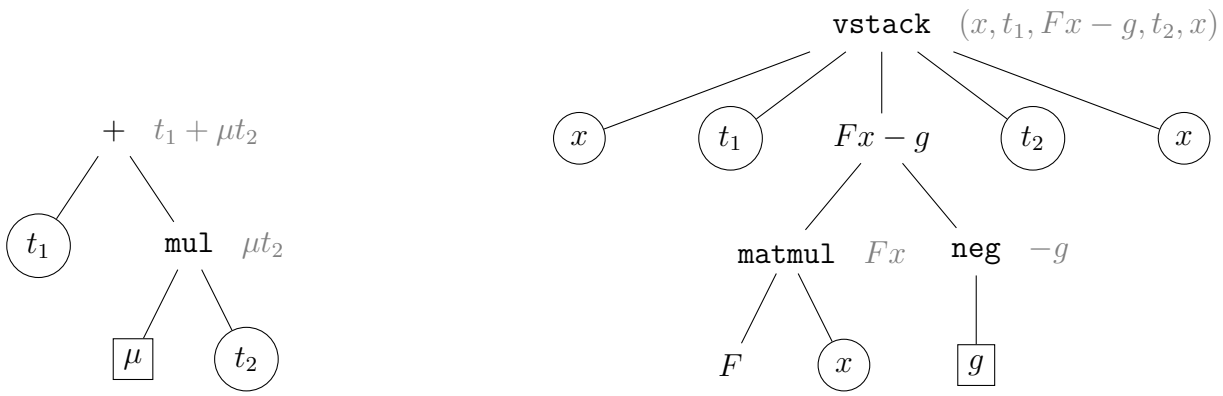
Figure 2.1 Users write functions in Python code that are restricted to the atom library. Variables are labeled with a circle, and parameters with a square.

Conic form expression tree. After applying the reductions, the problem has been *canonicalized* to

$$\begin{array}{ll} \text{minimize} & t_1 + \mu t_2 \\ \text{subject to} & x \in \mathbf{R}^n_+ \\ & (t_1, Fx - g) \in \mathcal{K}_{\text{soc},m+1} \\ & (t_2, x) \in \mathcal{K}_{\text{soc},n+1} \end{array} \tag{2.5}$$

where t_1 and t_2 are new epigraph variables, and the cone $\mathcal{K}$ from 1.1 is the product $\mathbf{R}^n_+ \times \mathcal{K}_{\text{soc},m+1} \times \mathcal{K}_{\text{soc},n+1}$. Problem (2.5) is now in the functional conic form introduced above: every constraint is a cone membership, and the objective and constraint expressions are biaffine. The expression trees of 2.2 are accordingly trees of biaffine operators. In the next section, we explain how these symbolic trees get converted into the parameter maps defined in 1.3.

References. Michael Grant's papers and PhD thesis on CVX and DCP are the standard references on verifying convexity and canonicalization [3, 4, 29]. The paper by Chu et al. [31] gives another perspective of canonicalization through the *relaxed smith form*, it was used extensively to learn the concept initially. The paper by Agrawal et al. [22] on rewriting sys-



(a) Expression tree of the objective

(b) Expression tree of the constraints

Figure 2.2 After the epigraph transformation reduction, the constraints and the objective are affine functions of the variables.

tems is a great reference for understanding reductions and the core concepts behind CVXPY. Finally, the `CVXPY` documentation and the Mosek modeling cookbook can be used as complementary references for epigraph reformulations and general modeling principles.

2.2 Matrix stuffing

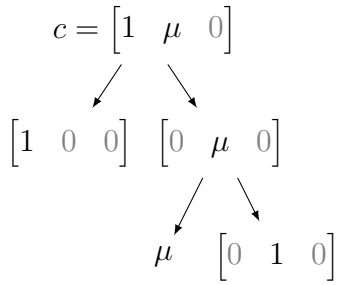
In this section, we explain how CVXPY forms the problem data from a set of affine expression trees. We first present the two compilation paths offered by CVXPY, which differ in how parameters are treated, and then cover fundamental data structures and design choices of the implementations, which can later be contrasted with the method proposed in 3. Historically, the implementations of matrix stuffing have been referred to as *canonicalization backends*, and there has been many versions over the years. We will focus on `cvxcore`, one of the original implementations in `C++`, and `SciPy`, an improved Python version which relies on the sparse matrix library with the same name.

Unified pathway. Historically, CVXPY maintained two separate canonicalization pathways: one targeting cone programs, with an affine objective and general cones, and one targeting quadratic programs, with a quadratic objective but only the zero and nonnegative cones. Motivated by modern solvers that accept conic constraints together with a quadratic objective, such as Clarabel [32], the two pathways were recently unified into the single form (2.2) [27]; QP solvers such as OSQP [33] and conic solvers such as ECOS [34] and MOSEK [35] consume its special cases.

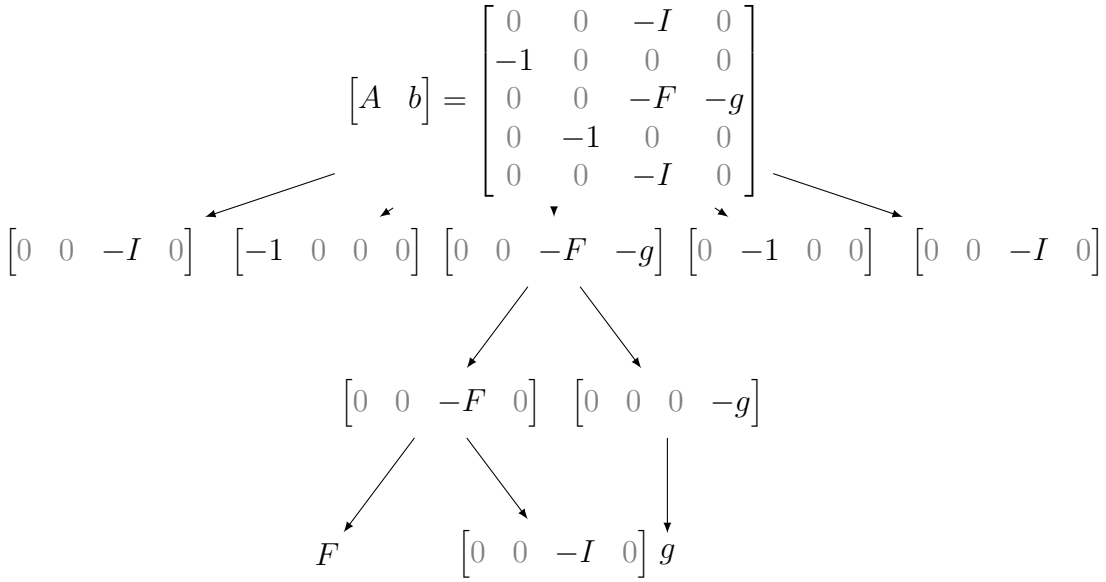
Two compilation paths. When a problem contains parameters, CVXPY offers two options. If the problem complies with the disciplined parameterized programming (DPP) ruleset, a mild restriction on how parameters may enter expressions, CVXPY compiles the problem once and caches the parameter maps of (1.3): the constraint map $\mathcal{R}$ is stored as a sparse matrix with one row per entry of $[A\ \ b]$ and one column per entry of $\tilde{\theta}$, so that re-solving for new parameter values only requires a matrix-vector product with $\tilde{\theta}$ followed by a reshape [26]. Subsequent solves bypass the reduction chain entirely and reuse the cached maps. The second option, selected by passing `ignore_dpp=True` (and also used as a fallback when a parameterized problem is not DPP-compliant), prepends a reduction called `EvalParams` which replaces every parameter node in the expression trees by a constant holding its current value. Canonicalization then proceeds on a parameter-free problem, but the entire reduction chain must be re-run every time a parameter changes. Figure 2.4 contrasts the two paths on the objective of our running example. This general path, which trades fast re-solves for unrestricted parameter expressions and a cheaper first compilation, is the focus of this thesis (3).

`Tensor` object. The word *tensor* is borrowed loosely here; we do not require the multilinear machinery of physics or differential geometry. We consider a `Tensor` as a three-dimensional array which represents a sparse linear mapping from a parameter vector to the constraint matrix (as shown in 1.3). The three indices of a `Tensor` are rows for constraints, columns for variables, and parameter offsets for parameters. Under the `ignore_dpp` path the parameter dimension is trivial: since all parameters have been evaluated to constants, the tensor has a single slice corresponding to the constant entry of $\tilde{\theta}$, and matrix stuffing reduces to forming the numerical data $[A\ \ b]$ directly.

Recursive walk. Fundamentally, matrix stuffing is relatively simple; it requires a single depth-first walk of a set of affine expression trees, where each tree represents a potentially vector-valued constraint. At the leaves of a tree, a variable maps to an identity matrix placed in the columns assigned to that variable, a constant maps to entries in the constant column, and a parameter maps to entries in its slices of the parameter dimension. Every internal node is one of roughly twenty-five linear operations (addition, negation, indexing, transpose, stacking, matrix multiplication, and so on), and each operation implements a recursive method which takes the already-stuffed `Tensor`s of its children — arbitrary in number and sparsity — applies its linear operation to each of them, and accumulates the results by addition. Since the process is recursive, every node of an expression tree will have an equivalent `Tensor`, and the final output (the `Tensor` at the root node) will give the constraint parameter map $\mathcal{R}$.



(a) Matrix stuffing of the objective

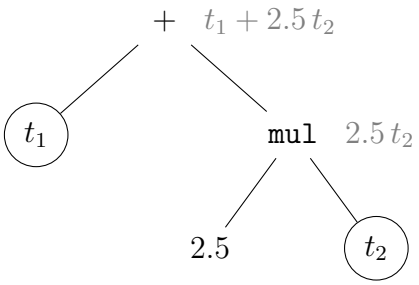


(b) Matrix Stuffing tree

Figure 2.3 Matrix stuffing of problem (2.4) by a recursive walk of the conic-form expression trees of 2.2: each node's matrix is obtained by applying a linear operation to the matrices of its children.

$$Q = \begin{array}{c} \\ t_1 \\ t_2 \\ x \\ d \end{array} \begin{array}{c} \begin{array}{ccc} g^T & \mu & 1 \end{array} \\ \begin{bmatrix} 0 \cdots 0 & 0 & 1 \\ 0 \cdots 0 & 1 & 0 \\ 0 \cdots 0 & 0 & 0 \\ 0 \cdots 0 & 0 & 0 \end{bmatrix} \end{array} \qquad \begin{bmatrix} c \\ d \end{bmatrix} = Q\,\tilde{\theta} = \begin{bmatrix} 1 \\ \mu \\ 0 \\ 0 \end{bmatrix}$$

(a) DPP path: the objective parameter map Q is formed and cached; each re-solve recovers the objective data (c, d) by multiplying Q with the extended parameter vector $\tilde{\theta} = (g, \mu, 1)$.



$$\Rightarrow \quad c = \begin{bmatrix} 1 & 2.5 & 0 \end{bmatrix}$$

(b) `ignore_dpp` path: `EvalParams` evaluates the parameters in the functional form, before canonicalization, so the canonicalized objective tree carries the constant 2.5 in place of the parameter leaf μ that the general path keeps (2.2). Canonicalizing this constant-data tree yields the numeric c directly, but the whole chain re-runs whenever a parameter changes.

Figure 2.4 The two compilation paths on the objective of problem (2.4): the DPP path caches the parameter map Q once and re-solves with a matrix-vector product, while the `ignore_dpp` path evaluates the parameters up front and re-canonicalizes from scratch.

Canonicalization backends. In `cvxcore`, the expression trees built in Python are first mirrored into equivalent C++ objects, and the recursive walk is performed in C++, where each node's `Tensor` is stored as triplet arrays (the coordinate format of 2.5) for every parameter slice. The `SciPy` backend is a more recent pure Python implementation of the same walk; it initially stored each node's `Tensor` as a dictionary mapping variables to lists of sparse matrix slices, one per parameter entry, which incurs a Python loop over the slices at every node. A *stacked-slices* reorganization removes most of these loops: the slices Q_i of a tensor are concatenated vertically into a single sparse matrix $\mathcal{S}(Q)$, and a linear operation L is applied to all slices at once through the Kronecker identity $\mathcal{S}(LQ) = (I_{p+1} \otimes L)\,\mathcal{S}(Q)$ [27]. A further variant replaces the `SciPy` matrices with a custom N-dimensional sparse coordinate representation, which is more memory-efficient for problems with many parameters and is the default for DPP problems with more than 1000 parameter entries. In all backends, each

constraint tree is processed independently with a running row offset, and the per-constraint results are concatenated into a single sparse matrix whose rows index the entries of $[A \;\; b]$ and whose columns index the entries of $\tilde{\theta}$ — the flattened constraint parameter map of (1.3), which degenerates to the single column $[A \;\; b]$ on the `ignore_dpp` path. The backend can be selected on a per-solve basis; `cvxcore` remains the default at the time of writing despite being officially deprecated in favor of the Python implementations.

References. The canonicalization backends are largely undocumented in the literature; the CVXPY and `cvxcore` source code are the primary references for this section, together with the rewriting system paper [22]. The DPP ruleset and the cached parameter maps were introduced in the paper on differentiable convex optimization layers [26].

2.3 Coefficient extraction

Matrix stuffing is CVXPY's particular answer to a task that every modeling language must perform: converting a symbolic model into the numerical problem data consumed by a solver. In this section, we review how three other modeling systems approach this *coefficient extraction* task and contrast their designs with CVXPY.

`JuMP`. JuMP is an algebraic modeling language embedded in Julia whose development started in 2012 at the MIT Operations Research Center by Iain Dunning, Joey Huchette, and Miles Lubin, with a first public release in 2013 [16,36]. It is now maintained by a team of core contributors, with Oscar Dowson as its most active maintainer. Unlike CVXPY, JuMP performs no convexity verification; it belongs to the derivative-based modeling paradigm, where the user is responsible for the properties of the formulation. Coefficient extraction in JuMP is *eager*: the `@constraint` and `@objective` macros rewrite the user's code at parse time into loops that build each affine expression as a sparse list of terms (an ordered dictionary mapping variables to coefficients). This macro rewriting avoids the asymptotic blowup of naive operator overloading, where summing d terms with repeated calls to `+` copies partial sums of growing size. As a consequence, the problem data is scalarized one constraint at a time while the model is being built, whereas CVXPY retains whole expression trees until solve time.

`MathOptInterface`. JuMP communicates with solvers through MathOptInterface (MOI), an abstraction layer whose standard form is to minimize $f_0(x)$ subject to constraints $f_i(x) \in S_i$, where the functions f_i and sets S_i are drawn from predefined lists [37]. There is de-

liberately no single canonical form: each solver advertises the function-set pairs it supports natively, and unsupported constraints are rewritten by *bridges*, local transformations selected per constraint type by a shortest-path search over a hypergraph of possible reformulations. Bridges play the same role as CVXPY's reductions, but they are chosen just-in-time as constraints are added to the model, rather than applied as a global pass over the whole problem. The constraint matrix only materializes inside the solver wrapper: when the model is copied to the solver, the affine terms of all constraints are stuffed into a single sparse matrix in compressed sparse column (CSC) format.

`AMPL.` AMPL is one of the original standalone algebraic modeling languages, developed by Robert Fourer, David Gay, and Brian Kernighan at Bell Laboratories and Northwestern University [10] to replace the error-prone matrix generators that practitioners previously wrote by hand [9]. An AMPL model is purely symbolic, with variables and constraints indexed over sets; the AMPL translator instantiates the model with data, applies a presolve, and writes the problem instance to a `.nl` file that is read by the solver [38]. The file splits each constraint into a linear part, stored as sparse coefficient lists along with the Jacobian sparsity pattern, and a nonlinear part, stored as an expression graph in Polish prefix notation. Inside the solver process, the AMPL Solver Library (ASL) reads the file, exposes the linear coefficients (in CSC format if the solver requests it), and evaluates the nonlinear Jacobian entries with reverse mode algorithmic differentiation (AD) on the expression graphs [39, 40].

`CasADi.` CasADi is a software framework for nonlinear optimization and optimal control built around algorithmic differentiation [17]. Expressions are stored in two graph types, scalar-valued `SX` nodes and matrix-valued `MX` nodes, and every matrix in the framework is sparse in CSC format. CasADi takes coefficient extraction to its logical extreme: for nonlinear programs it never forms a coefficient matrix at all, and instead differentiates the expression graph to generate callback functions for the objective gradient, the sparse constraint Jacobian, and the Hessian of the Lagrangian. The sparsity patterns of these matrices are detected by propagating bitvectors through the graph, and their values are recovered in few AD sweeps using the graph coloring techniques covered in 2.4. For quadratic programs, however, CasADi does extract explicit matrices, differentiating the symbolic objective and constraints to identify the data H, A and their sparsity patterns.

Comparison. Despite their different designs, these systems share a common thread: preparing the problem data amounts to computing the sparse Jacobian of the constraint functions with respect to the variables. JuMP accumulates it eagerly, one scalarized constraint at

a time; AMPL writes it to a file and defers the nonlinear entries to AD inside the solver; CasADi only generates functions that evaluate it; and CVXPY stuffs it from canonicalized affine trees, with the additional map from parameters to problem data. This connection motivates the next section, which reviews how derivatives are computed efficiently.

References. The MathOptInterface paper [37] is the best reference on the design of JuMP's solver abstraction layer and its bridge system. David Gay's technical reports [38, 39] remain the authoritative description of the `.nl` format and the AMPL solver interface. Finally, the CasADi paper [17] is a self-contained reference covering its expression graphs, AD implementation, and solver interfaces.

2.4 Computing derivatives

Finding the rate of change or the *derivative* of a function at a particular point is a fundamental concept in calculus that was discovered by Leibniz and Newton in the late 1600s. Since then, although the underlying theory has practically remained the same, various methods have been developed to efficiently compute derivatives in different contexts and applications. More recently, the rise of machine learning and deep learning brought renewed interest in this age-old problem through a new application of derivatives in training *neural networks*, it is known as the *backward pass* or *backpropagation* [41, 42]. Some theorists will claim that backpropagation is little more than a clever application of the *chain rule* [43], but the large scale of modern workloads has led to the development of incredible software tools and implementation techniques which are interesting in their own right. This section provides an overview of these new tools and techniques, with a particular emphasis on *algorithmic differentiation* (AD) and its applications to computing sparse Jacobians [44].

Numerical differentiation. Other than computing derivatives by hand, the simplest method is to use *numerical differentiation*, which approximates the derivative of a function at a point by evaluating the function at nearby points using a *finite difference* formula similar to either

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \tag{2.6}$$

or

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}, \tag{2.7}$$

where h is a small step size. Although this approach is straightforward to implement and can achieve reasonable accuracy, there are important downsides to consider. First, it requires a

linear number of function evaluations with respect to the number of input variables, which is expensive for high-dimensional functions. Second, it can easily lead to *truncation errors* (when h is too large) or *rounding errors* (when h is too small) in the computed derivative if the step size is not carefully chosen [45]. Despite these downsides, numerical differentiation is still widely used in practice, especially for verifying the correctness of derivative computations (also known as a *derivative checker*) [46].

Symbolic differentiation. Mathematical expressions can be manipulated automatically using simple derivative rules of calculus, such as the product and sum rule. This *symbolic* approach to differentiation is used in computer algebra systems such as Mathematica [47] and Maple [48] as well as the SymPy library in Python [49]. Unlike its numerical counterpart, symbolic differentiation tools compute derivatives as closed-form expressions, which can be simplified and evaluated at any arbitrary point. These closed-form expressions are particularly useful for deriving analytic solutions of optimization problems, and can be further used in mathematical proofs and analyses.

However, a naive implementation of symbolic differentiation can easily lead to exponentially large symbolic expressions, which is impractical and inefficient for many problems [44]. Interestingly, when intermediate expressions are stored as *common subexpressions*, and the expression is represented with a *directed acyclic graph* (DAG) instead of a tree, symbolic differentiation performs the same set of operations as *reverse mode differentiation* [50].

Algorithmic differentiation. There is a lot of confusion on the terminology surrounding the various methods for computing derivatives; *automatic differentiation* and *computational differentiation* are often used to describe the field, but we will instead refer to it as *algorithmic differentiation* since it emphasizes the algorithms behind the different implementations rather than a particular method (it is also the convention in the seminal book by Griewank and Walther [44]). Fundamentally, AD is a non-standard interpretation of a computer program: it augments the *evaluation trace* of a function (the sequence of floating point values calculated during the execution) with calculations of its derivatives [51]. This trace can be represented as a *computational graph*, a DAG in which each node corresponds to an elementary operation with known derivatives. Combining the derivatives of these operations throughout the graph using the chain rule gives the derivative of the overall function [44]. The computational graph is the central data structure of AD; the various *modes* of AD correspond to different ways elementary derivatives are accumulated on this graph.

Forward mode AD. The method of propagating tangents forward through the computational graph is known as *forward mode* AD. A single forward pass simultaneously computes the value and derivative of every intermediate variable with respect to a single input variable, giving the sensitivity of all outputs to that input. The cost of a forward pass is a small constant factor of the cost of evaluating the original function, since each elementary operation is augmented with at most a few additional arithmetic operations for the derivatives. However, one downside of forward mode AD is that the number of forward passes scales linearly with the number of input variables, which is impractical for certain functions (which have many input variables and very few output variables).

Reverse mode AD. In contrast to forward mode, *reverse mode* AD propagates adjoints backward through the computational graph. It first requires a forward pass to compute and store the value of every intermediate variable, and it then performs a reverse pass to compute the sensitivities of a single output variable with respect to all input variables. The cost of a reverse pass is also a small constant factor of the cost of evaluating the original function, but the number of passes required now scales linearly with the number of output variables rather than the number of input variables. This makes reverse mode AD particularly efficient for computing gradients of scalar-valued functions, making it the default method for modern machine learning libraries such as Jax [52], Pytorch [53], and Tensorflow [54].

Advantages of AD. [55].

Jacobian and Hessian computation.

Sparse Jacobian computation. graph coloring, cross-country elimination, vertex elimination, jacobian accumulation, etc. maybe mention graph coloring here?

[56] presents a code generator for operations on sparse data.

[57] presents a similar method for computing sparse Hessians with graph coloring. [58] is a survey of graph coloring methods for computing sparse Jacobians and Hessians.

Differentiation packages. mention ADOL-C and CppAD for C++. mention AMPL and CasADi for optimization. and also mention that they were covered above in the coeff extraction section.

References. The book by Griewank and Walther [44] is a comprehensive reference on the theory and practice of automatic differentiation. The book by Naumann [55] is another great reference that focuses on the implementation of AD in software, with a particular emphasis on the use of operator overloading and source code transformation techniques. [51] is an excellent and relatively up-to-date survey of the various methods for automatic differentiation in machine learning.

2.5 Sparse matrices

Matrices that contain many number of zero entries are known as *sparse matrices*. The simplest sparse matrix data structure is the *coordinate* or *triplet* format. It consists of two integer index arrays `rows` and `cols` and a floating point value array `vals` of length equal to the number of *nonzero entries* in the matrix.

For example, the matrix

$$\begin{bmatrix} 0 & 0 & 3.2 & 0 \\ 4.1 & 1.8 & 0 & 2.2 \\ 0 & 2.8 & 0 & 3.4 \end{bmatrix} \tag{2.8}$$

can be represented in coordinate format as

```
int    rows[] = {0,  1,   1,   1,   2,   2};
int    cols[] = {2,  0,   1,   3,   1,   3};
double vals[] = {3.2, 4.1, 1.8, 2.2, 2.8, 3.4};
```

where we use zero-based indexing and array declaration notation from the C programming language [59]. The coordinate format is the easiest to understand and form, but it is not effective for most operations on sparse matrices; the *compressed sparse row* or CSR format is commonly used instead.

Compressed sparse row format. The CSR format differs from the coordinate format in that it replaces the `rows` array with a row pointer array `idxptr` of length equal to the number of rows plus one. The row pointer array indicates the starting index of each row in the `cols` arrays. For example, the matrix in (2.8) can be represented in CSR format as

```
int  idxptr[] = {0,  1,              4,        6};
int   cols[] = {2,  0,   1,   3,   1,   3};
double vals[] = {3.2, 4.1, 1.8, 2.2, 2.8, 3.4};
```

where we deliberately format `idxptr` to align with the start of each row in the `cols` arrays. The benefits of the CSR format are more than a reduced size of the `idxptr` array compared

to the `rows` array. In the CSR format, accessing a row takes constant time and is very *cache-friendly* since the nonzero entries are stored contiguously in memory. For this reason, the CSR format is the default for *iterative methods* to solve sparse linear systems [60].

Compressed sparse column format. However, one downside of the CSR format is that accessing a column is much more difficult; one would need to look at every row to find the nonzero entries in a column. An alternative format is the *compressed sparse column* or CSC format, which swaps the role of the `idxptr` array to refer to column indices instead. In contrast to iterative methods, the CSC format is the preferred format for *direct methods* to solve sparse linear systems [61].

Transpose.

Which format to use?

Other formats. The coordinate, CSR and CSC formats are the most widely used sparse matrix formats, but many others exist for different specialized use cases. The scientific Python sparse library `scipy.sparse` [62] allows users to form other types of sparse matrices such as the list of lists (LIL) matrix, the dictionary of keys (DOK) matrix, and the block-sparse row (BSR) matrix.

Sparse matrix struct in C. A CSR matrix struct in C looks like this.

```
typedef struct CSR_Matrix {
    int *p;      /* row pointers, size m+1 */
    int *i;      /* column indices, size nnz */
    double *x;   /* values, size nnz       */
    int m, n, nnz;
} CSR_Matrix;
```

References. Although the work in this thesis does not necessarily revolve around solving sparse linear systems, a lot of techniques for manipulating sparse matrices we use were inspired by the literature. In particular, the book by Davis [61] was used extensively throughout the development of our methods. The book by Duff [63] is another great reference to learn more about the history and development of sparse matrix algorithms. Another reference is the review article by the "gang of four" (Gill, Murray, Saunders and Wright) from

the Systems Optimization Laboratory at Stanford University [64] which provides a high-level overview of various applications of sparse matrix methods in optimization.

CHAPTER 3 A lightweight canonicalization engine

This chapter presents a minimal implementation of a subset of affine atoms in the `SparseDiffEngine` package, and shows how they compute the sparse constant Jacobian of a set of affine expressions. We start from a *canonicalized* problem in standard form, produced by the canonicalization of §2.1. The goal is the same as in §2.2: form the problem data. §3.2 describes the data structures, §3.3 the atom implementations, and §3.6 the problem class and its CVXPY interface. We assume the sparse-matrix background of §2.5.

3.1 Keeping parameters, splitting the walk

Matrix stuffing already walked the objective tree of our running example. Figure 2.3 annotated each node of the canonicalized objective $t_1 + \mu t_2$ of (2.5) with the row of c it produces, ending in $c = \begin{bmatrix} 1 & \mu & 0 \end{bmatrix}$ at the root. The engine walks the very same tree of 2.2, but it differs from matrix stuffing in two ways.

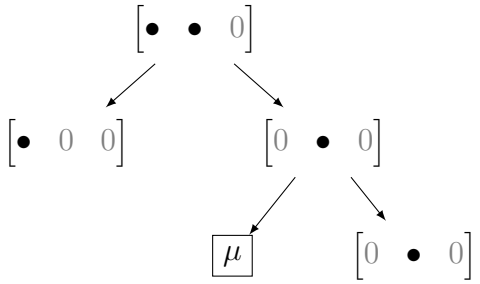
First, it keeps the parameter. The `ignore_dpp` path of §2.2 begins with `EvalParams`, which deletes the parameter leaf μ and replaces it by a constant holding its current value (2.4b); the walk that follows is parameter-free and must be redone from scratch whenever μ changes. We instead leave μ in place as a leaf node, so the tree that produces c is fixed once and reused across solves.

Second, it splits the single walk into two passes over that fixed tree, shown in 3.1. A *structural* pass fixes the sparsity pattern of every node's Jacobian from the patterns of its children, independent of the value of μ (3.1a). A *value* pass then fills the numbers into that fixed pattern, reading μ's current value at the leaf (3.1b). The structural pass runs once per problem family; only the value pass repeats on each solve, whereas matrix stuffing fuses the two and recomputes both on every change of μ. These passes are the `jacobian_init` and `eval_jacobian` functions that the rest of the chapter develops atom by atom.

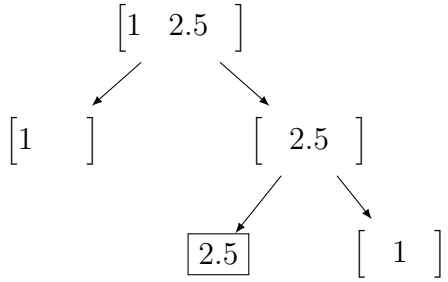
On this objective the saving is admittedly hard to see: the structural pass over $t_1 + \mu t_2$ is trivial, and one could fairly object that re-evaluating so small a tree from scratch costs nothing. The split earns its keep where the structural work is both expensive and shared across solves. The clearest case in our running example is the constraint tree of 2.3, which contains the product Fx (3.2). The Jacobian of Fx is a sparse matrix–matrix product handled by the matrix multiplication atom of §3.3, and its $-F$ block has a sparsity *pattern* that is the costly part to derive and is independent of any parameter value. Because the parameter

g enters the same constraint, the `ignore_dpp` path re-derives this block from scratch each time g changes, while the engine fixes its pattern once in the structural pass (3.2a) and only refills values (3.2b). The per-solve cost this removes, negligible for the scalar objective but substantial for problems with large sparse products solved repeatedly, is quantified in §4.

Like `cvxcore`, which mirrors the Python expression trees into equivalent C++ objects, the engine mirrors each CVXPY atom onto a C counterpart and rebuilds the canonicalized trees as the C `expr` trees of §3.2; both passes then run entirely in C.



(a) Structural pass: a bullet marks a structural nonzero.



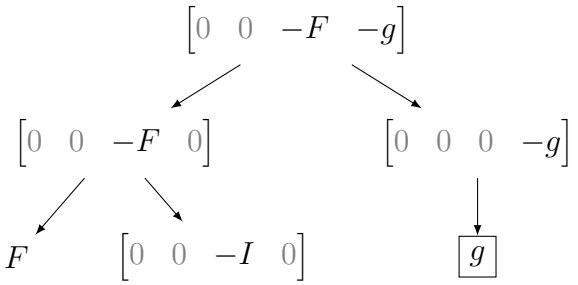
(b) Value pass: entries refilled here for $\mu = 2.5$.

Figure 3.1 The engine's two passes over the objective tree of 2.2, with the parameter leaf μ kept across both.

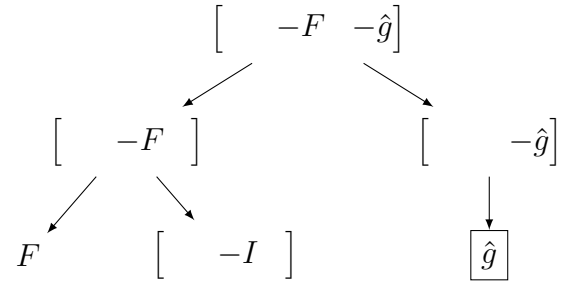
3.2 Fundamental data structures

The main data structure in the engine is the expression struct.

```
typedef struct expr {
    int   d1, d2, size; /* shape (d1 x d2) and size = d1*d2 */
    int   n_vars;       /* total number of variables     */
    int   var_id;       /* offset into u, or NOT_A_VARIABLE */
```



(a) Structural pass: the matmul block $-F$ inherits the pattern of F and is independent of g.



(b) Value pass: only the values are written into that pattern, here for $g = \hat{g}$. Between solves only the parameter block $-\hat{g}$ changes; the matmul block $-F$ is untouched.

Figure 3.2 The two passes on the $Fx - g$ subtree of the constraint tree of 2.3. The constant F enters through the matmul atom, whose block $-F$ has a sparsity pattern fixed once; across solves only the parameter g changes.

```
    struct expr *left;   /* left child (NULL for leaves)  */
    struct expr *right;  /* right child (NULL for leaves) */

    double    *value;    /* forward-pass output, size entries */
    CSR_Matrix *jacobian; /* size x n_vars sparse Jacobian */

    forward_fn     forward;
    jacobian_init_fn jacobian_init_impl;
    eval_jacobian_fn eval_jacobian;
} expr;
```

Each node carries a `value`, the output of its forward pass, and a sparse Jacobian in CSR form. Each node also implements three functions: `forward`, `jacobian_init`, and `eval_jacobian`.

Leaf nodes. The leaves are variables and parameters.

3.3 Atom implementations

An atom takes one or two expression operands and produces an expression. Each atom implements the three functions: `forward` computes the value, `jacobian_init` fixes the Jacobian's sparsity pattern once, and `eval_jacobian` fills its values on each solve. We give the full C implementation of one atom, `add`, to establish the pattern, and describe the rest in prose, each anchored by its CVXPY expression. Throughout, `x` and `y` denote variables, `a` and `b` parameters, and `F` a constant matrix, all of compatible shape.

Addition. The addition atom sums two expressions elementwise.

```
add_expr = cp.add(x, y)
```

The implementation below is the template every other atom follows. The forward pass evaluates the children and sums their values. `jacobian_init` allocates the output Jacobian, whose number of nonzeros is at most the sum of the children's, and fixes its pattern. `eval_jacobian` sums the children's Jacobians into it.

```
static void forward(expr *node, const double *u)
{   /* forward pass for children */
    node->left->forward(node->left, u);
    node->right->forward(node->right, u);
    for (int i = 0; i < node->size; i++)
    {
        node->value[i] = node->left->value[i] + node->right->value[i];
    }
}

static void jacobian_init_impl(expr *node)
{
    /* initialize children's jacobians */
    jacobian_init(node->left);
    jacobian_init(node->right);

    /* we never have to store more than the sum of children's nnz */
    int nnz_max = node->left->jacobian->nnz + node->right->jacobian->nnz;
    node->jacobian = new_csr_matrix(node->size, node->n_vars, nnz_max);

    /* fill sparsity pattern */
    sum_csr_alloc();
```

```
}

static void eval_jacobian(expr *node)
{
    /* evaluate children's jacobians */
    node->left->eval_jacobian(node->left);
    node->right->eval_jacobian(node->right);

    /* sum children's jacobians */
    sum_csr_fill_values();
}
```

Negation. The negation atom negates an expression elementwise. The forward pass flips the sign of each value. The Jacobian has the child's pattern, so `jacobian_init` copies it and `eval_jacobian` negates its values.

Transpose. The transpose atom transposes an expression.

```
transpose_expr = cp.transpose(x)
```

The forward pass writes the transpose into the row-major output buffer. Transposing only relabels the output entries, so the Jacobian is a fixed row permutation of the child's: `jacobian_init` records the permutation, and `eval_jacobian` copies the child's values into the permuted rows.

Horizontal stack. The horizontal stack atom concatenates expressions of equal row count side by side.

```
hstack_expr = cp.hstack([x, y])
```

The forward pass concatenates the arguments' values. The Jacobian stacks the arguments' Jacobian rows: `jacobian_init` lays out the combined pattern, and `eval_jacobian` copies each argument's values contiguously into place.

Vertical stack. The vertical stack atom concatenates expressions of equal column count.

```
vstack_expr = cp.vstack([x, y])
```

It needs no new code. The engine uses the identity `vstack([A, B]) = transpose(hstack([transpose(A) transpose(B)]))`, inheriting the forward and Jacobian behaviour of transpose and hstack.

Indexing. The indexing atom selects a subset of an expression's entries.

```
index_expr = x[0:3]
```

The forward pass gathers the selected entries. The Jacobian is the corresponding subset of the child's rows: `jacobian_init` records which rows are selected, and `eval_jacobian` copies their values.

Sum. The sum atom reduces all entries of an expression to a scalar.

```
sum_expr = cp.sum(x)
```

The forward pass accumulates the child's values into a scalar. The Jacobian is the column-wise sum of the child's Jacobian, a single row: `jacobian_init` fixes its pattern, and `eval_jacobian` accumulates the child's values into it.

Elementwise multiplication. The elementwise multiplication atom scales an expression entrywise by data `a`, constant or parameter.

```
elemwise_mult_expr = cp.multiply(x, a)
```

The forward pass multiplies the child's values by `a`, refreshing `a` first if it is a parameter. Scaling row i by a_i leaves the child's pattern unchanged, so `jacobian_init` copies it and `eval_jacobian` scales each entry of row i by a_i. A change in `a` is absorbed entirely at evaluation.

Matrix multiplication. The matrix multiplication atom takes a constant matrix $A \in \mathbf{R}^{m\times k}$ and an expression x with $x \in \mathbf{R}^k$, and produces $Ax \in \mathbf{R}^m$. The canonicalized problem is affine, so one operand is always data; we take it to be the left operand A, with the variable-bearing expression as the right child.

```
matmul_expr = A @ x
```

The forward pass computes the sparse matrix–vector product $y = Ax$. If x has Jacobian $J_x \in \mathbf{R}^{k\times n_{\text{vars}}}$, then y has Jacobian $J_y = AJ_x$, a sparse matrix–matrix product. Its pattern depends only on the patterns of A and J_x.

This is the one atom where the two-phase split does real work. We factor the product into a *symbolic* routine, which allocates J_y and fills its pattern from the patterns of A and J_x, and a *numeric* routine, which fills the values into that pattern. `jacobian_init` runs the symbolic routine once; `eval_jacobian` runs only the numeric routine on each solve. The symbolic

step is the expensive part of a sparse matrix–matrix product, and it is hoisted out of the per-solve path.

3.4 Problem class

The atoms compute the Jacobian of a single expression. A `Problem` ties together the full set of constraint expression trees and assembles their root Jacobians into the global problem data $[A\ b]$ and objective data $[c\ d]$.

At construction the problem runs the one-time *initialization* phase. It walks each expression tree once, calling `jacobian_init` bottom-up so every node allocates its Jacobian and fixes its pattern. The pattern of an affine expression is independent of its parameter values, so this walk runs once per problem family. The root Jacobians are concatenated into the global CSR buffer for A using the constraint row offsets of §2.2, fixing the pattern of the problem data as a whole.

3.5 Parameter updates

The *evaluation* phase runs on every solve. Given new parameter values θ, the problem refreshes the leaf parameters and calls `eval_jacobian` bottom-up, filling the values of each pre-allocated Jacobian without touching its pattern. The forward pass supplies the constant column b, and the root Jacobians are copied into the global buffer. A parameter update therefore costs only a forward pass and a numeric Jacobian evaluation: no allocation, no symbolic work, no graph mutation.

This is the saving over the `ignore_dpp` path of §2.2. That path is general — it places no DPP restriction on how parameters enter — but re-derives the symbolic structure on every change of θ. Our two-phase design keeps that generality while paying the symbolic cost once, recovering the re-solve speed of the cached DPP path without materializing the three-dimensional parameter tensor.

3.6 CVXPY interface

We expose the `Problem` class and the atom constructors through a thin set of Python bindings. The engine then acts as a *canonicalization backend*, occupying the place of `cvxcore` and the `SciPy` backend of §2.2: it consumes the canonicalized affine expression trees from the reductions and returns the problem data (A, b, c, d).

CVXPY hands the backend its canonicalized expression trees. The binding mirrors each

CVXPY atom onto its engine counterpart, builds the C expression trees, runs the initialization phase once, and stores the handle on the problem. On each solve with new parameter values, CVXPY calls the evaluation phase to obtain the updated data, which is forwarded to the cone solver. The binding forwards only pointers to pre-allocated buffers, so no problem data is copied between Python and C in the inner loop.

CHAPTER 4 Benchmark results

In this chapter we evaluate the engine of §3 against CVXPY's existing canonicalization path. The central question is the cost of repeatedly forming the problem data for a parameterized problem family, which is exactly the regime our two-phase design targets.

4.1 Benchmark Setup

We benchmark on the public CVXPY benchmark suite [65], 25 real-world problems (linear and quadratic programs, least squares, portfolio models, robust regression, and conic and semidefinite programs) maintained under *airspeed velocity* (`asv`) [66] with seeded, reproducible data. Three problems need the unsupported `kron` atom and are excluded; the remaining 22 run on both backends.

We time *canonicalization* only—the construction of the problem data (A, b, c, P) from the expression tree—and exclude the numerical solve, since that is what the engine changes. Concretely we time `get_problem_data` with the target solver fixed to Clarabel [32]: the `CPP` (`cvxcore`) baseline is the default DPP path, and the C engine is selected with `ignore_dpp`.

```
prob.get_problem_data(solver=cp.CLARABEL)              # CPP baseline (DPP
    tensor)
prob.get_problem_data(solver=cp.CLARABEL, ignore_dpp=True) # our C engine
```

We report two regimes: *cold-start* (a fresh `Problem` built with nothing cached, averaged over 3 builds) and *warm-restart* (parameter values reassigned on a warm cached chain, averaged over 20 re-compiles)—the latter being the regime the engine targets. Alongside these wall-clock times we report *peak memory*, and summarize across the suite with the *geometric mean* of the per-problem engine/`CPP` ratios, the standard scale-free statistic for timings that span orders of magnitude [67].

All measurements run single-threaded on a MacBook Pro (13-inch, M2, 2022): 8-core Apple M2, 24 GB RAM, macOS 26.3, with Python 3.14, CVXPY 1.10 (dev), NumPy 2.4 [30], and SciPy 1.16 [62]. Versions are pinned and the suite and engine are public, so the configuration is reproducible.

4.2 Results across the benchmark suite

Table 4.1 reports cold-start compilation across the 22 runnable benchmarks, and Table 4.2 reports warm-restart re-compilation on the parametric subset. In both, the ratio is C engine/CPP, so a value below 1 means the engine is faster; the final row gives the geometric mean of the per-problem ratios.

Table 4.1 Cold compilation time: a fresh problem formed and canonicalized from scratch, C engine versus CVXPY's `CPP` backend (22 runnable benchmarks). Ratio = C engine / `CPP`; values below 1 favor the engine.

Benchmark	`CPP` (ms)	C engine (ms)	Ratio
Yitzhaki	845	69	**0.08**
Cajas	2154	179	**0.08**
SimpleQP	2882	295	0.10
LeastSquares	2313	263	0.11
HuberRegression	2559	298	0.12
SimpleLP	5656	747	0.13
SlowPruning	1726	283	0.16
SVMWithL1Regularization	2593	411	0.16
TvInpainting	879	164	0.19
SimpleScalarParametrizedLP	1066	223	0.21
SimpleFullyParametrizedLP	463	110	0.24
ParametrizedQP	2645	631	0.24
ConeMatrixStuffing	2589	819	0.32
ParamSmallMatrixStuffing	118	40	0.34
OptimalAdvertising	2202	805	0.37
ParamConeMatrixStuffing	69	27	0.38
SmallMatrixStuffing	1633	627	0.38
CVaR	7128	6224	0.87
FactorCovarianceModel	1157	1117	0.97
Murray	1052	1627	1.55
ConvexPlasticity	81	220	2.63
SemidefiniteProgramming	995	8886	**8.90**
Geometric mean			**0.32**

Reading. In aggregate the engine is about 3× faster on cold builds (geometric-mean ratio 0.32) and near parity on parametric re-compiles (1.18), but the per-problem split is the more informative story. On *cold* compilation the engine wins on the bulk of the LP, QP, portfolio, and conic problems—typically 3–12× faster, since it extracts the problem data directly from

Table 4.2 Parametric re-compilation time on a warm cached chain: parameter values are reassigned and the problem data rebuilt (the engine's target regime). Ratio = C engine / `CPP`.

Benchmark	#par	`CPP` (ms)	C engine (ms)	Ratio
ParamSmallMatrixStuffing	3	2.23	0.65	**0.29**
ParamConeMatrixStuffing	3	1.19	0.40	0.33
SimpleFullyParametrizedLP	1	50.8	36.8	0.72
ParametrizedQP	2	333	309	0.93
SimpleScalarParametrizedLP	1	66.2	67.9	1.03
FactorCovarianceModel	2	21.3	72.2	3.39
SVMWithL1Regularization	1	40.2	150	3.72
ConvexPlasticity	3	43.0	194	4.52
Geometric mean				**1.18**

the expression trees while `CPP` pays the full parameter-tensor build. It loses on `Murray` (1.55×), `ConvexPlasticity` (2.63×), and the `SemidefiniteProgramming` outlier (8.9×), and reaches parity on `FactorCovarianceModel` (0.97×). On *parametric re-compilation* the picture is mixed and use-case dependent: the engine is faster on the matrix-stuffing and fully-parametrized-LP re-solves (0.29–0.72×) and near parity on `ParametrizedQP`, but 3.4–4.5× slower on `FactorCovarianceModel`, `SVM`, and `ConvexPlasticity`, where `CPP`'s cached DPP parameter→data tensor (one sparse matmul per solve) is hard to beat. The largest memory wins come from keeping constants sparse: eliminating an eager $25\,000 \times 25\,000$ dense densification (≈ 5 GB) on `FactorCovarianceModel`, and shrinking the `ConvexPlasticity` Hessian block from 9M to 51k nonzeros by routing a mostly-zero `quad_form` matrix to the sparse binding.

4.3 Scaling with problem size

The cross-suite tables fix the problem size; here we instead sweep it. We deep-dive three parameterized problems—parametric least squares, a factor-covariance portfolio, and an energy-storage scheduling model—and re-measure cold-start time, warm-restart time, and peak memory as the size grows. All three are genuinely parametric, so the warm-restart curve measures the engine's target regime: refilling values on a fixed sparsity pattern as the parameters change. For each panel we plot `CPP` against the C engine on log–log axes.

4.3.1 Parametric least squares

The bread-and-butter case: fit x to noisy observations with the data matrix A and right-hand side b supplied as parameters, so a re-solve only reassigns A and b. The size knobs are the number of observations m and variables n.

```
import cvxpy as cp

A = cp.Parameter((m, n))    # data matrix  (reassigned each solve)
b = cp.Parameter(m)         # observations (reassigned each solve)

x = cp.Variable(n)
objective  = cp.Minimize(cp.sum_squares(A @ x - b))
constraints = [0 <= x, x <= 1]
prob = cp.Problem(objective, constraints)
```

We sweep $n \in \{250, 500, 1000, 2000, 4000\}$ with $m = 3n$ (overdetermined) and measure both backends in the *ignore-DPP* regime: the canonicalization is rebuilt from scratch whenever the parameters change, which is the engine's intended use case. The C engine carries the parameters natively and re-extracts values on a fixed sparsity pattern; the `CPP` baseline substitutes the parameter values (the `EvalParams` reduction) and re-canonicalizes the resulting constant-data problem. We deliberately avoid CPP's DPP parameter-tensor path here: because the data matrix A is itself an $m \times n$ parameter, that tensor has $\Theta(mn)$ entries and its construction dominates everything else—tens of seconds and gigabytes already at $n = 500$—so it is neither representative nor the comparison of interest. Cold-start and warm-restart times are medians over three repetitions; peak memory is the process resident-set high-water mark (`ru_maxrss`), measured in a fresh subprocess per point so that, unlike Python-level `tracemalloc`, it also captures the engine's C-side allocations.

Cold-start (Figure 4.1) favours the engine by roughly 4× across the range (ratio 0.21–0.27), both backends scaling as $\sim n^{2.1}$. The gap widens on warm-restart (Figure 4.2): `CPP` re-canonicalizes in essentially its cold-start time (9.1 s at $n = 4000$), whereas the engine re-extracts on the cached pattern in 863 ms—a 10–12× speed-up that is precisely the non-DPP re-solve regime the engine targets. Peak memory (Figure 4.3) is comparable: both grow as $\sim n^{1.6}$ and are dominated by the dense A, with the engine 2–30% heavier in resident set. Its own internal accounting (the engine's `verbose` "peak memory") grows as $\sim n^{2.0}$, from 14 MB at $n = 250$ to 3.7 GB at $n = 4000$, yet stays below the process RSS throughout.

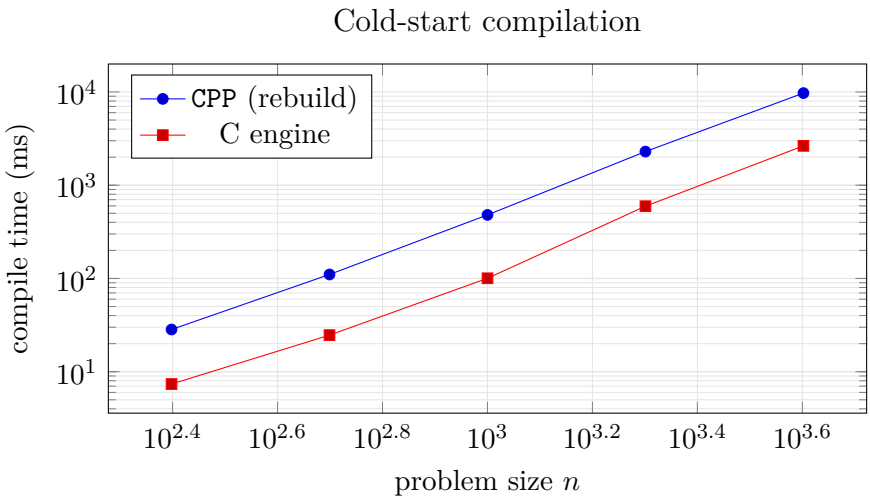


Figure 4.1 Parametric least squares, cold-start: full canonicalization from scratch versus problem size n ($m = 3n$). The C engine is $\approx 4\times$ faster across the whole range; both backends scale as $\sim n^{2.1}$.

4.3.2 Factor-covariance portfolio

A Markowitz portfolio whose risk is expressed through a low-rank factor model: n assets load on $m \ll n$ factors with covariance Σ, plus idiosyncratic diagonal risk D. The risk-aversion γ and leverage limit $L_{\max}$ are parameters, so re-solving traces the efficient frontier without rebuilding the problem. The size knobs are the number of assets n and factors m.

```
import cvxpy as cp
import numpy as np

# mu: expected returns, F: factor loadings (n x m),
# Sigma: factor covariance (m x m), D: idiosyncratic variances (diagonal).
w = cp.Variable(n)                # portfolio weights
f = cp.Variable(m)                # factor exposures
gamma = cp.Parameter(nonneg=True) # risk aversion (reassigned each solve)
Lmax  = cp.Parameter()            # leverage limit (reassigned each solve)

ret  = mu.T @ w
risk = cp.quad_form(f, Sigma, assume_PSD=True) + cp.sum_squares(np.sqrt(D) @ w
    )
objective  = cp.Maximize(ret - gamma * risk)
constraints = [cp.sum(w) == 1, f == F.T @ w, cp.norm(w, 1) <= Lmax]
prob = cp.Problem(objective, constraints)
```

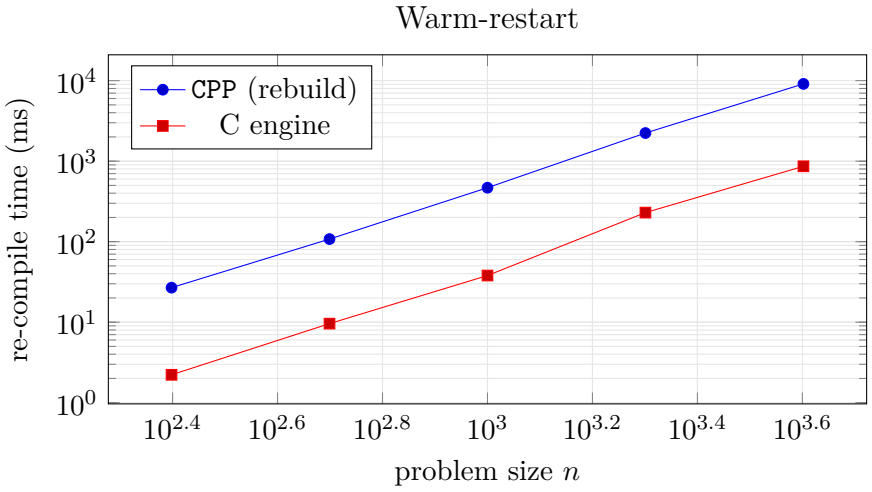


Figure 4.2 Parametric least squares, warm-restart: rebuild after reassigning A and b. With no DPP tensor to reuse, `CPP` pays its full cold-start cost again, while the C engine only re-evaluates values on the cached sparsity pattern—a 10–12× speed-up.

We hold the factor count fixed at $m = 100$ and sweep the assets $n \in \{500, 1000, 2000, 4000, 8000\}$. As in the least-squares study, both backends run in the *ignore-DPP* rebuild regime—the engine carries $\gamma, L_{\max}$ natively and re-extracts on a fixed sparsity pattern, while `CPP` substitutes their values and re-canonicalizes. (Here the parameters are scalar, so CPP's DPP cache would in fact be feasible; we use the same rebuild regime as before for a consistent comparison.) Times are medians over three repetitions; peak memory is the process resident-set high-water mark measured in a fresh subprocess.

Cold-start (Figure 4.4) again favours the engine, but less decisively than for least squares and with a narrowing margin: from 0.41× at $n = 500$ to 0.88× at $n = 8000$, as the engine's $\sim n^{1.2}$ cold cost catches CPP's $\sim n^{0.9}$. Warm-restart (Figure 4.5) is where the engine dominates: `CPP` pays its full cold-start cost on every re-solve (144 ms at $n = 8000$), while the engine refills the cached pattern in 10.7 ms—a steady 12–13× advantage. Peak memory (Figure 4.6) is close, the engine 4–16% heavier in resident set; both curves are nearly flat because the low-rank model keeps the operator small. The engine's own internal peak (its `verbose` accounting) grows as $\sim n^{0.9}$, from 6 MB at $n = 500$ to 74 MB at $n = 8000$.

4.3.3 Energy-storage scheduling

A simple energy problem: schedule a battery over a horizon of T hours to minimize the cost of grid energy, where the hourly price is the parameter that changes between solves. The state of charge is the running sum of charge minus discharge, and the battery must end

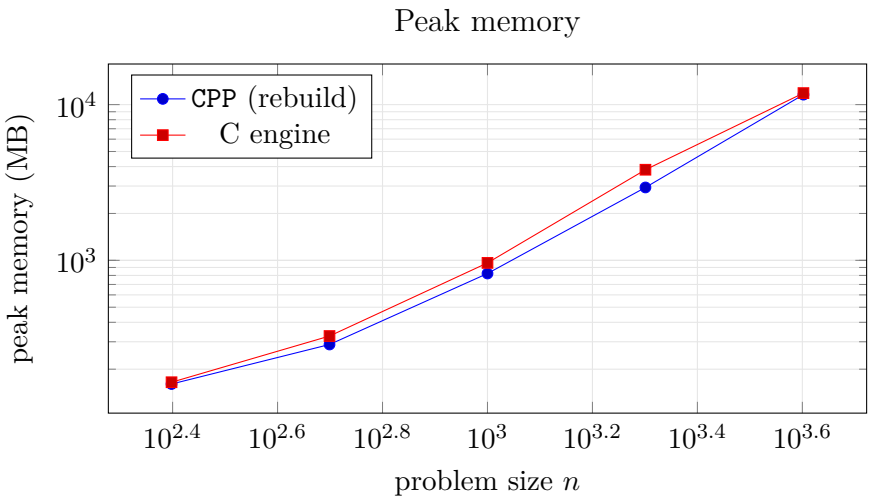


Figure 4.3 Parametric least squares, peak process memory (resident-set high-water mark). The backends are within ∼30% of each other; both grow as $\sim n^{1.6}$, dominated by the shared interpreter and the dense data matrix A.

empty. The size knob is the horizon T.

```
import cvxpy as cp
import numpy as np

price = cp.Parameter(T)          # hourly price (reassigned each solve)
load  = np.random.rand(T)        # fixed demand profile
Q, Pmax = 50.0, 20.0             # capacity and charge/discharge power limit

charge    = cp.Variable(T, nonneg=True)
discharge = cp.Variable(T, nonneg=True)
soc = cp.cumsum(charge - discharge)     # battery state of charge

constraints = [charge <= Pmax, discharge <= Pmax,
               soc >= 0, soc <= Q, soc[T - 1] == 0]
grid = load + charge - discharge        # net energy drawn from the grid
prob = cp.Problem(cp.Minimize(price @ grid), constraints)
```

The cross-suite results (§4.2) and these scaling curves together delineate the operating regime where the engine pays off: cheap from-scratch builds across the suite, and a favorable size trend on problems whose canonicalization cost the engine avoids re-paying.

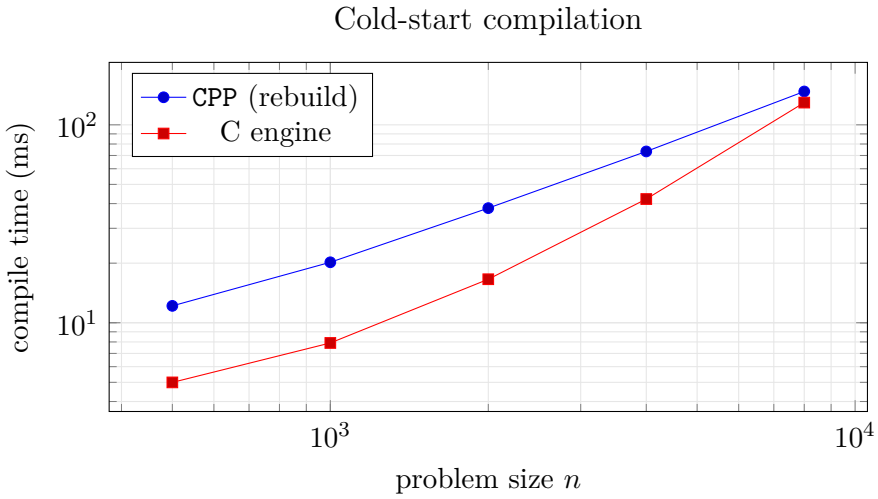


Figure 4.4 Factor-covariance portfolio, cold-start versus number of assets n ($m = 100$ factors). The engine leads at small n but its advantage shrinks with size—it scales as $\sim n^{1.2}$ against CPP's $\sim n^{0.9}$, reaching near-parity by $n = 8000$.

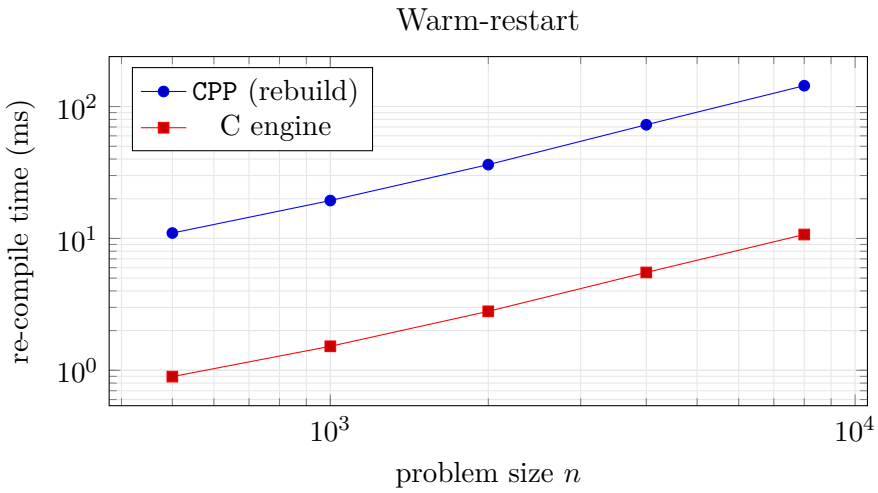


Figure 4.5 Factor-covariance portfolio, warm-restart after reassigning γ and $L_{\max}$. `CPP` re-canonicalizes in essentially its cold-start time; the engine re-extracts on the cached pattern, a stable 12–13× speed-up across the range.

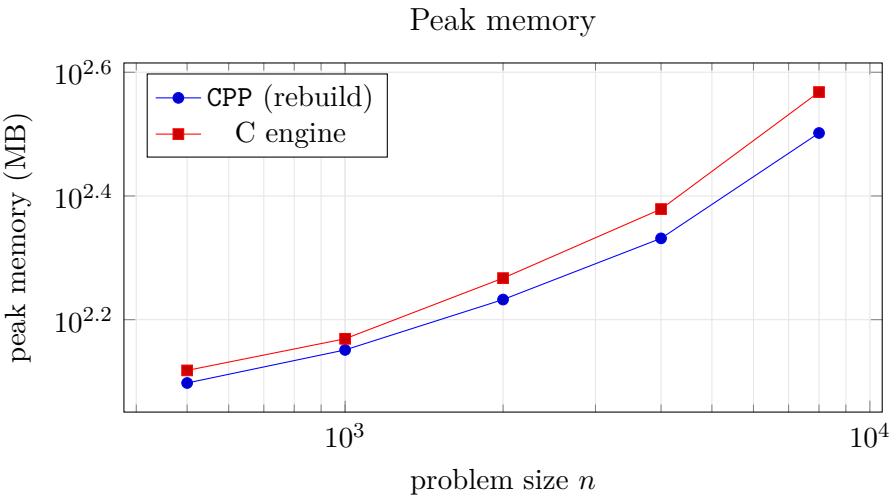


Figure 4.6 Factor-covariance portfolio, peak process memory. The backends are within $\sim$15%; both grow slowly ($\sim n^{0.35}$), reflecting the low-rank factor structure rather than a dense operator.

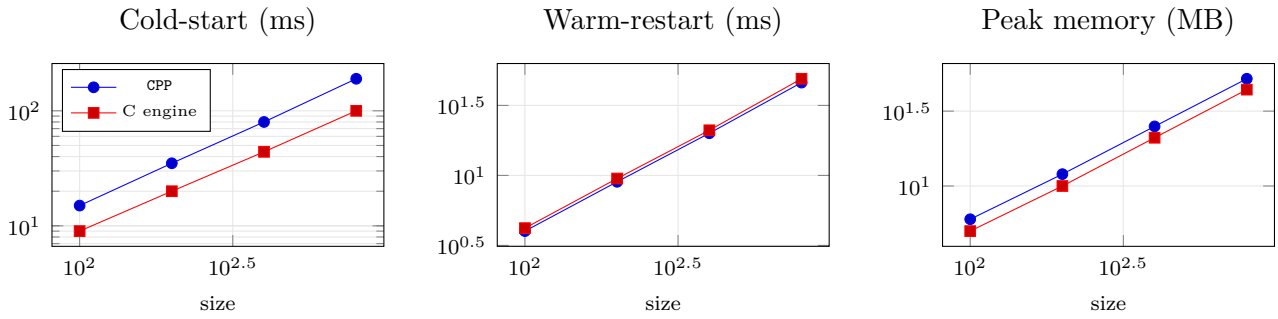


Figure 4.7 Energy-storage scheduling: scaling of cold-start, warm-restart, and peak memory with horizon length T (*placeholder curves; data pending the size-sweep run*).

CHAPTER 5 CONCLUSION

5.1 Summary of Works

5.2 Limitations

5.3 Future Research

REFERENCES

[1] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[2] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE, 2004, pp. 284–289.

[3] M. Grant, "Disciplined Convex Programming," Ph.D. dissertation, Stanford University, Dec. 2004, ph.D. dissertation. [Online]. Available: https://web.stanford.edu/~boyd/papers/pdf/mcg_thesis.pdf

[4] M. Grant, S. Boyd, and Y. Ye, "Disciplined convex programming," in *Global Optimization: From Theory to Implementation*, L. Liberti and N. Maculan, Eds. New York: Springer, 2006, pp. 155–210.

[5] M. Schaller, G. Banjac, S. Diamond, A. Agrawal, B. Stellato, and S. Boyd, "Embedded code generation with cvxpy," *IEEE Control Systems Letters*, vol. 6, pp. 2653–2658, 2022.

[6] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," 1978.

[7] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[8] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[9] R. Fourer, "Modeling languages versus matrix generators for linear programming," *ACM Trans. Math. Softw.*, vol. 9, no. 2, pp. 143–183, Jun. 1983. [Online]. Available: https://doi.org/10.1145/357456.357457

[10] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[11] J. Bisschop, *AIMMS optimization modeling*, 2006.

[12] A. Brooke, D. Kendrick, and A. Meeraus, *GAMS: A User's Guide*. Redwood City, CA: The Scientific Press, 1988.

[13] W. Orchard-Hays, "History of mathematical programming systems," *Annals of the History of Computing*, vol. 6, no. 3, pp. 296–312, 1984.

[14] M. L. Bynum *et al.*, *Pyomo—Optimization Modeling in Python*. Springer, 2021.

[15] W. E. Hart *et al.*, "Pyomo: modeling and solving mathematical programs in python," *Mathematical Programming Computation*, vol. 3, no. 3, pp. 219–260, 2011.

[16] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[17] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[18] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[19] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," https://cvxr.com/cvx, Aug. 2012.

[20] A. Fu, B. Narasimhan, and S. Boyd, "Cvxr: An r package for disciplined convex optimization," *Journal of Statistical Software*, vol. 94, no. 14, 2020. [Online]. Available: http://dx.doi.org/10.18637/jss.v094.i14

[21] M. Udell, K. Mohan, D. Zeng, J. Hong, S. Diamond, and S. Boyd, "Convex optimization in Julia," in *2014 first workshop for high performance technical computing in dynamic languages*. IEEE, 2014, pp. 18–28.

[22] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[23] A. Agrawal, S. Diamond, and S. Boyd, "Disciplined geometric programming," *Optimization Letters*, vol. 13, pp. 961–976, 2019.

[24] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, pp. 1643–1657, 2020.

[25] D. Cederberg, W. Zhang, P. Nobel, and S. Boyd, "Disciplined nonlinear programming," 2025.

[26] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and Z. Kolter, "Differentiable convex optimization layers," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[27] W. Zhang, P. Nobel, A. Jeendgar, R. Murray, P. Schiele, and S. Diamond, "CVXPY 1.9: Recent advances in optimization modeling software," https://www.cvxpy.org/_static/cvxpy-1.9.pdf, 2026, preprint.

[28] E. M. B. Smith, "On the optimal design of continuous processes," Ph.D. dissertation, Imperial College London (University of London), London, 1996, ph.D. dissertation.

[29] M. Grant and S. Boyd, "Graph Implementations for Nonsmooth Convex Programs," in *Recent Advances in Learning and Control*, ser. Lecture Notes in Control and Information Sciences, V. Blondel, S. Boyd, and H. Kimura, Eds. Springer, London, 2008, vol. 371. [Online]. Available: https://doi.org/10.1007/978-1-84800-155-8_7

[30] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: https://doi.org/10.1038/s41586-020-2649-2

[31] E. Chu, N. Parikh, A. Domahidi, and S. Boyd, "Code generation for embedded second-order cone programming," in *2013 European Control Conference (ECC)*, 2013, pp. 1547–1552.

[32] P. J. Goulart and Y. Chen, "Clarabel: An interior-point solver for conic programs with quadratic objectives," *arXiv preprint arXiv:2405.12762*, 2024.

[33] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: an operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.

[34] A. Domahidi, E. Chu, and S. Boyd, "ECOS: An SOCP solver for embedded systems," in *European Control Conference (ECC)*, 2013, pp. 3071–3076.

[35] M. ApS, *MOSEK optimization suite 11.0*, 2026.

[36] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[37] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, "Mathoptinterface: a data structure for mathematical optimization problems," 2020. [Online]. Available: https://arxiv.org/abs/2002.03447

[38] D. M. Gay, "Writing .nl files," Sandia National Laboratories, Tech. Rep. SAND2005-7907P, 2005, describes AMPL's expression DAG and coefficient extraction internals.

[39] ——, "Hooking your solver to AMPL," Bell Laboratories, Tech. Rep. 97-4-06, 1997, reference for the AMPL Solver Library (ASL).

[40] ——, "Automatic differentiation of nonlinear AMPL models," AT&T Bell Laboratories, Tech. Rep. Numerical Analysis Manuscript 91-05, 1991, reverse mode AD on AMPL expression graphs.

[41] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[42] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. [Online]. Available: https://epubs.siam.org/doi/10.1137/16M1080173

[43] Y. LeCun, "A theoretical framework for back-propagation," in *Proceedings of the 1988 Connectionist Models Summer School*, vol. 1. CMU, Pittsburgh, Pa, 1988, pp. 21–28.

[44] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2008.

[45] P. C. Chakravarti, "Truncation error in interpolation and numerical differentiation," *Numer. Math.*, vol. 3, no. 1, p. 279–284, Dec. 1961. [Online]. Available: https://doi.org/10.1007/BF01386029

[46] P. Wolfe, "Checking the calculation of gradients," *ACM Trans. Math. Softw.*, vol. 8, no. 4, p. 337–343, Dec. 1982. [Online]. Available: https://doi.org/10.1145/356012.356013

[47] Wolfram Research, Inc., "Mathematica, Version 14.3," 2026, champaign, IL. [Online]. Available: https://www.wolfram.com/mathematica

[48] Maplesoft, a division of Waterloo Maple Inc., "Maple," Waterloo, Ontario. [Online]. Available: https://www.maplesoft.com

[49] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, v. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, "Sympy: symbolic computing in python," *PeerJ Computer Science*, vol. 3, p. e103, Jan. 2017. [Online]. Available: https://doi.org/10.7717/peerj-cs.103

[50] S. Laue, "On the equivalence of forward mode automatic differentiation and symbolic differentiation," *ArXiv*, vol. abs/1904.02990, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:102350450

[51] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *J. Mach. Learn. Res.*, vol. 18, no. 1, p. 5595–5637, Jan. 2017.

[52] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[53] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: an imperative style, high-performance deep learning library.* Red Hook, NY, USA: Curran Associates Inc., 2019.

[54] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "Tensorflow: a system for large-scale machine learning," in *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'16. USA: USENIX Association, 2016, p. 265–283.

[55] U. Naumann, *The Art of Differentiating Computer Programs: An Introduction to Algorithmic Differentiation*, ser. Software, Environments, and Tools. Philadelphia, PA: SIAM, 2011.

[56] P. Herholz, X. Tang, T. Schneider, S. Kamil, D. Panozzo, and O. Sorkine-Hornung, "Sparsity-specific code optimization using expression trees," *ACM Trans. Graph.*, vol. 41, no. 5, May 2022. [Online]. Available: https://doi.org/10.1145/3520484

[57] A. H. Gebremedhin, A. Tarafdar, A. Pothen, and A. Walther, "Efficient computation of sparse hessians using coloring and automatic differentiation," *INFORMS J. on Computing*, vol. 21, no. 2, p. 209–223, Apr. 2009. [Online]. Available: https://doi.org/10.1287/ijoc.1080.0286

[58] A. H. Gebremedhin, F. Manne, and A. Pothen, "What color is your Jacobian? Graph coloring for computing derivatives," *SIAM Review*, vol. 47, no. 4, pp. 629–705, 2005.

[59] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[60] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2003.

[61] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2006. [Online]. Available: https://epubs.siam.org/doi/book/10.1137/1.9780898718881

[62] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[63] I. S. Duff, A. M. Erisman, and J. K. Reid, *Direct Methods for Sparse Matrices*. Oxford, England: Oxford University Press, 1986.

[64] P. E. Gill, W. Murray, M. A. Saunders, and M. H. Wright, "Sparse matrix methods in optimization," *SIAM Journal on Scientific and Statistical Computing*, vol. 5, no. 3, pp. 562–589, 1984.

[65] CVXPY developers, "CVXPY benchmarks," https://github.com/cvxpy/benchmarks, 2025.

[66] airspeed velocity developers, "airspeed velocity," https://asv.readthedocs.io/, 2025, python benchmarking framework.

[67] D. Cederberg and S. Boyd, "Presolving for GPU-accelerated first-order LP solvers," https://web.stanford.edu/~boyd/papers/pslp.html, 2026, manuscript.